Precision Unified Automation Framework

Team : **The Locators**
Project Source code : Github

Document	**Page Number**
Architecture Documentation	02
Execution Reports	07
Team roles and responsibilities	08

Architecture Documentation

This project is a Hybrid Test Automation Framework designed to validate applications using UI (Selenium) and API (REST Assured) testing within a single ecosystem. It supports parallel execution, BDD (Cucumber), and rich reporting using Allure.The framework follows a modular, scalable, and maintainable architecture using Maven multi-module design.

The Framework designed to be

- Scalable
- Maintainable
- Reusable
- Parallel Execution Ready

Test cases Automated :

Application Under Test: Automation Exercise

Phase 1: Web UI Automation

● TC1: Register New User and validate
● TC2: Valid Login
● TC3: Invalid Login
● TC4: Add Product to the Cart and validate
● TC5: Remove Item from cart and validate

Phase 2: API Automation .

● TC 1. Get All Products List GET : {url}/api/productsList.
● TC 2. Get Single Product Details POST: {url} /api/searchProduct (using search_product parameter)
● TC 3. Create/Register User Account POST {url}//api/createAccount
● TC 4. Delete User Account DELETE {url}/api/deleteAccount
● TC 5: Update User Account PUT {url}/api/updateAccount
● TC 6. Negative Validation

Phase 3: The Hybrid Model [UI +API]

Module Structure

```
Hybrid_Automation_Framework
│
├── common
│   ├── config
│   ├── driver
│   ├── utils
│   └── resources
│
├── ui-test
│   ├── pages
│   ├── stepDefinitions
│   ├── runner
│   └── features
│
├── api-test
│   ├── client
│   ├── requestBuilder
│   ├── tests
│   ├── validator
│   └── resources
│
├── hybrid-test
│   ├── client
│   ├── pages
│   ├── stepDefinitions
│   ├── runner
│   └── features
│
└── reports (Allure Results)
```

Tech Stack Used

Selenium
RestAssured
TestNG
Cucumber
Allure Reports
Log4j

Core Components

1. Common Module (Shared Utilities)

- driver

Uses ThreadLocal for parallel execution

- config

Environment configuration (URLs, browser, etc.)

- utils

Wait utilities
Screenshot utilities
Logger utilities

2. UI Test Module

- pages

Implements Page Object Model (POM)
Encapsulates UI locators and actions

- stepDefinitions

Maps feature steps to UI actions

- runner

Executes tests using Cucumber + TestNG

3. API Test Module

- client

Handles API calls (GET, POST, DELETE)

- requestBuilder

Builds request payloads dynamically

- validator

Validates API responses (status, schema, data)

- tests

Contains API-specific test logic

4. Hybrid Test Module

Combines API + UI in a single scenario

Enables:
API → Create data
UI → Validate data
UI → Perform actions
API → Re-validate

Flow:
API → Create User
UI → Login using same user
UI → Delete user
API → Validate user deletion

Execution Flow

Feature File (BDD)
↓
Step Definitions
↓
Service Layer (UI / API / Hybrid)
↓
DriverManager / ApiClient
↓
Application Execution
↓
Validation (Assertions)
↓
Allure Reporting

Parallel Execution Strategy

Implemented using TestNG + Cucumber

Enabled via:
@DataProvider(parallel = true)
thread-count in testng.xml
WebDriver managed using ThreadLocal

Reporting (Allure Reports)

Captures:
Test steps
Logs
Screenshots (on failure)
Execution status

Categorization using:
Epic
Feature
Story

Design Patterns Used

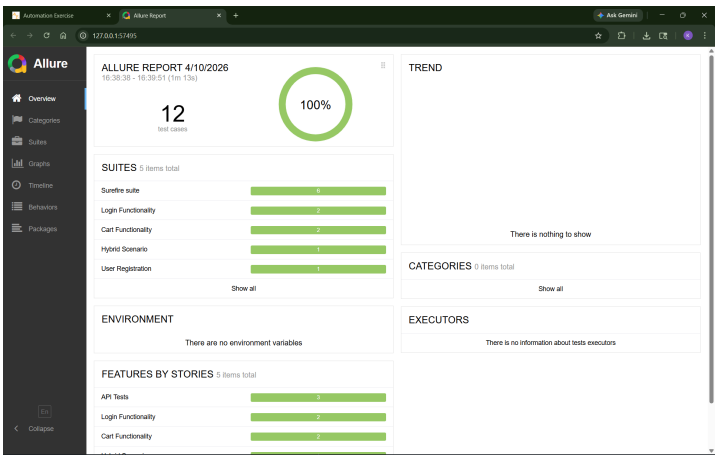
Pattern	Usage
Page Object Model	UI layer
Builder Pattern	API request creation
Singleton (DriverManager)	WebDriver management
Factory Pattern	Driver initialization
BDD (Cucumber)	Test design

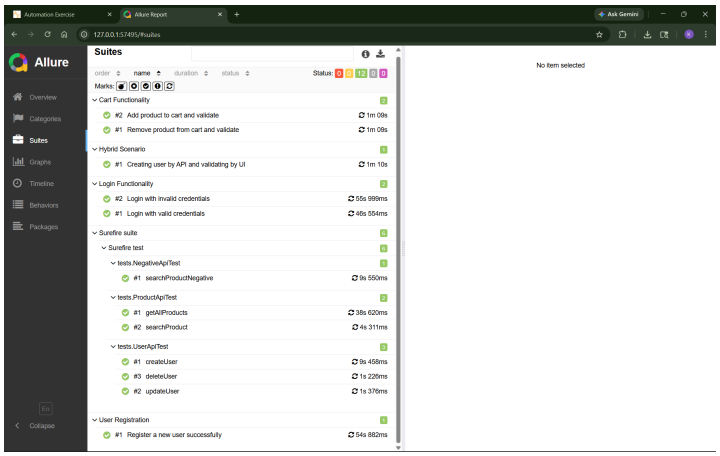
Test Types Supported

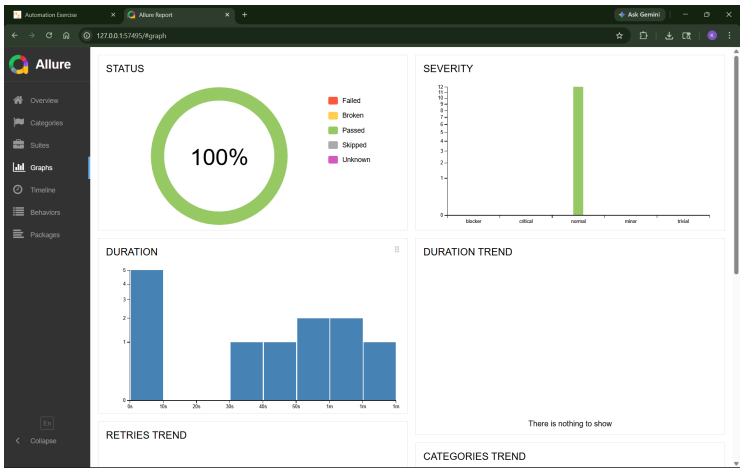
UI Automation Tests
API Automation Tests
Hybrid End-to-End Tests

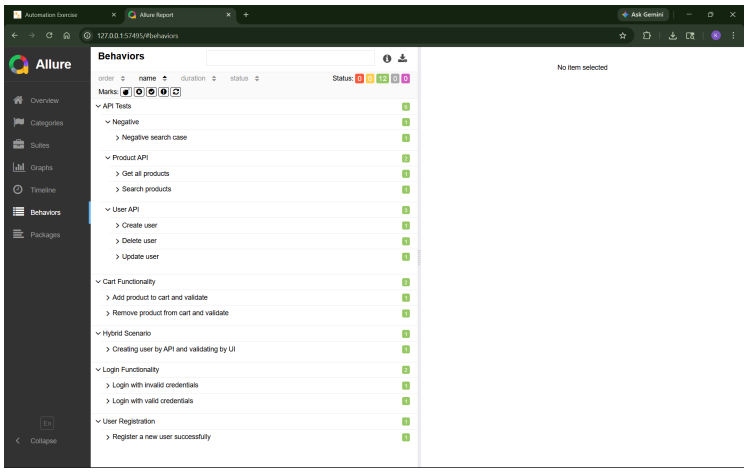
Execution Reports

```
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for Hybrid_Automation_Framework 1.0-SNAPSHOT:
[INFO]
[INFO] Hybrid_Automation_Framework ........................ SUCCESS [  0.187 s]
[INFO] common ............................................. SUCCESS [  1.704 s]
[INFO] ui-test ............................................ SUCCESS [01:13 min]
[INFO] api-test ........................................... SUCCESS [01:16 min]
[INFO] hybrid-test ........................................ SUCCESS [01:14 min]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  01:18 min (Wall Clock)
[INFO] Finished at: 2026-04-10T16:39:52+05:30
[INFO] ------------------------------------------------------------------------
```









Team Roles & Responsibilities

This project was developed collaboratively by a team of four members. Each member owned a core module while also contributing across the entire framework to ensure consistency, integration, and quality.

- **Nagendra Babu** – Project Architecture & Common Module

Designed the overall project structure using Maven multi-module architecture
Developed the common module, including:
WebDriver management (ThreadLocal-based)
Configuration management
Utility classes (Waits, Logger, Screenshot, etc.)

- **Aakash** – UI Automation Module

Built the UI automation framework using Selenium and Cucumber
Implemented Page Object Model (POM) for maintainable UI interactions
Developed step definitions and feature files for UI test scenarios
Integrated UI tests with TestNG parallel execution

- **John Prakesh** – API Automation Module

Developed the API automation framework using REST Assured
Implemented:
API client layer (GET, POST, DELETE operations)
Request builders for dynamic payload creation
Validators for response validation (status, schema, data)

- **Karthikeya Venkata Sai Vinayak** – Hybrid Automation Module

Designed and implemented the Hybrid testing module, combining UI and API workflows
Created end-to-end scenarios:
API → Data creation
UI → Validation & actions
API → Re-validation
Ensured data sharing between API and UI layers using thread-safe mechanisms